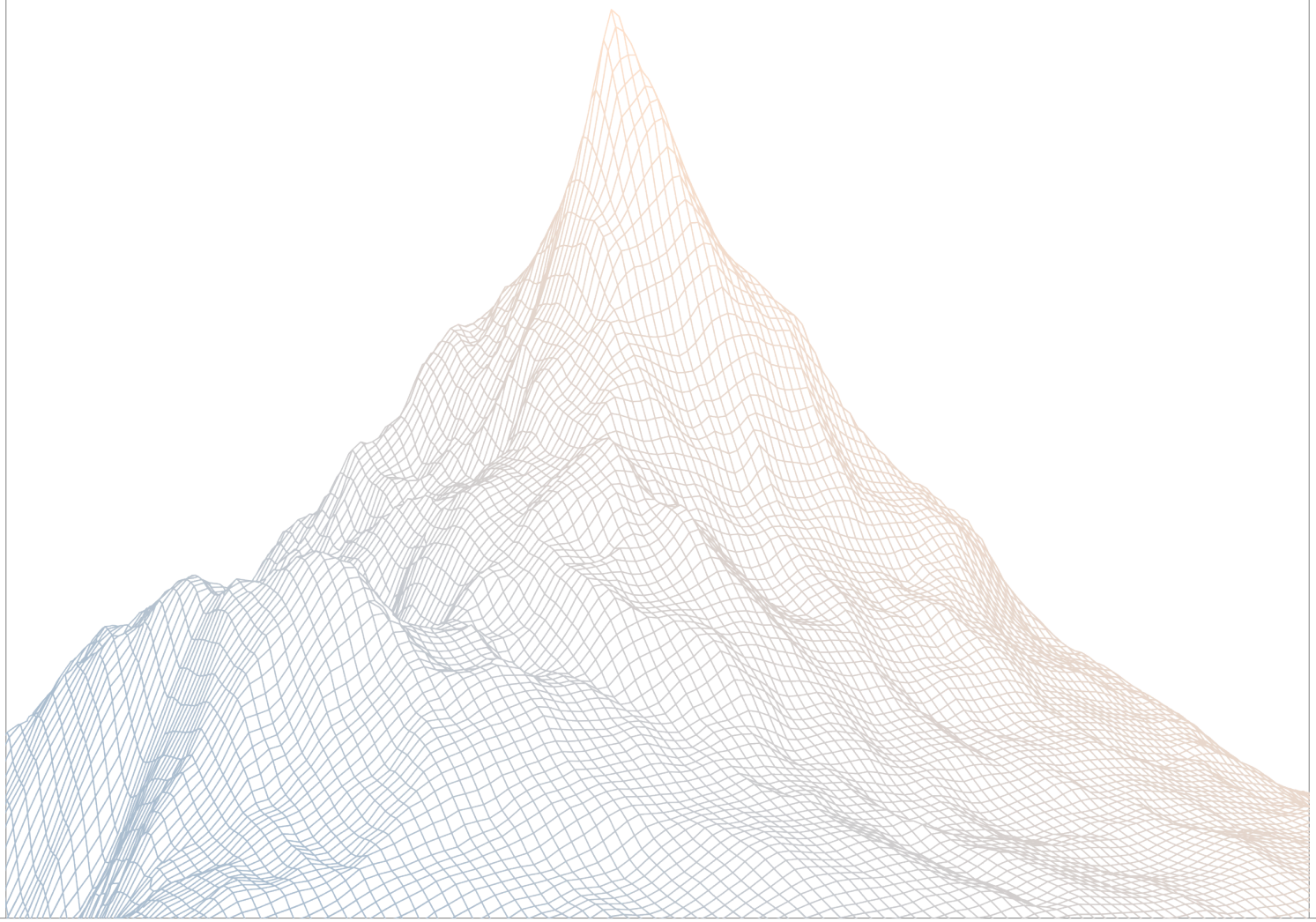
Zenith

City Protocol

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES: June 18th to June 25th, 2026

AUDITED BY: adriro
ether_sky

Contents

1 Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at https://zenith.security.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About City Protocol

City Protocol is an open infrastructure for neofinance, powered by tokenization, vaults, and modular financial services.

Our vision is to empower global access to high-quality yield assets and full-stack neofinance services onchain.

Financial products are moving onto crypto rails, but the infrastructure for launching them is still fragmented.

2.2 Scope

The engagement involved a review of the following targets:

Target	venzo-contracts
Repository	https://github.com/CityProtocol/venzo-contracts.git
Commit Hash	`12fa17b32260ad49b3e827bda577f61ecdb044f6`
Files	`src/**/*.sol` `Excluding the following: src/deferred/`

Target	Mitigation Review
Repository	https://github.com/CityProtocol/venzo-contracts.git
Commit Hash	`8eca1fef1fd972573ac5692d4e08d4fa2b4bb473`
Files	`src/**/*.sol` `Excluding the following: src/deferred/`

2.3 Audit Timeline

June 18, 2026	Audit start
June 25, 2026	Audit end
June 29, 2026	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	1
High Risk	0
Medium Risk	12
Low Risk	12
Informational	9
Total Issues	**34**

3

Findings Summary

ID	Description	Status
C-1	Cancelled redeem queue entries can permanently lock later redemptions	Resolved
M-1	The calculation of assetsPerShare and activeSupply in _consumeSettlementReport is incorrect	Resolved
M-2	The time gap between pricing and settlement can lead to unexpected changes in the share price	Resolved
M-3	The redeem epoch should be priced using its start time rather than its closing time	Resolved
M-4	Redeem settlement can regress the NAV cache	Resolved
M-5	Active supply is under-derived for non-18-decimal assets	Resolved
M-6	Share transfers do not validate blocked senders	Resolved
M-7	Deposits can use expired cached NAV	Resolved
M-8	The openedAt should be reset when all epochs are cancelled	Resolved
M-9	There could be a conflict between the strategy manager and the oracle report	Resolved
M-10	The calculation of the management fee and performance fee is incorrect	Resolved
M-11	The rounding direction is incorrect when depositing into the vault	Resolved
M-12	The fee is not correct minted when settling epochs	Resolved
L-1	Redemption pricing excludes accrued management and performance fees	Resolved
L-2	Redeem max views ignore pause and access policy state	Resolved
L-3	maxDeposit() and maxMint() can return unusable limits	Resolved

ID	Description	Status
L-4	maxDeposit() and maxMint() do not mirror caller access checks	Resolved
L-5	The initialNavAssets should be stored in a cache	Acknowledged
L-6	Unclaimable dust shares may permanently remain in the vault	Acknowledged
L-7	Users can adjust their deposits based on the upcoming report	Acknowledged
L-8	_convertToAssetsAt() truncates share value before applying price	Resolved
L-9	Redeem cancellation bypasses pause and current access checks	Resolved
L-10	The highWaterMarkAssetsPerShare monotonically increases over time	Resolved
L-11	The share token should not be rescuable	Resolved
L-12	An older report can be recorded as the latest report	Resolved
I-1	Cancelled redeems can return shares without current transfer eligibility	Resolved
I-2	If settlement is not executed for a long period, it may cause future settlements to be reverted	Acknowledged
I-3	Fee manager replacement is not emitted	Resolved
I-4	Idle assets and cached NAV must be reconciled across reports	Acknowledged
I-5	Fee rate updates can apply retroactively to unsettled fees	Acknowledged
I-6	Vault interface detection omits AccessControl support	Resolved
I-7	Settlement report timestamp parameter is misleading	Resolved

ID	Description	Status
I-8	OffchainStrategyManager allocation can diverge from real strategy value	Acknowledged
I-9	Redeem allowance spending remains available while share transfers are disabled	Resolved

4

Findings

4.1 Critical Risk

A total of 1 critical risk findings were identified.

[C-1] Cancelled redeem queue entries can permanently lock later redemptions

SEVERITY: Critical	IMPACT: High
STATUS: Resolved	LIKELIHOOD: High

Target

- RedeemEpoch.sol
- AsyncRedeemVault.sol

Description:

`cancelRedeemRequest()` clears the controller's shares for the current redeem epoch, but `_cancelRedeemRequest()` does not remove the request id from `redeemRequestQueue` or advance `nextRedeemRequestIndex`. Later, `redeem()`, `withdraw()`, `maxRedeem()`, and `maxWithdraw()` resolve claimable requests through `_currentRedeemRequest()`, which only reads the queue item at the current cursor and never skips settled zero-share requests. If the cursor points to a cancelled request whose epoch has settled, `_claimableRedeemRequestState()` reverts because `request.shares == 0`, so the controller cannot reach any later settled request in the queue.

This can permanently block a controller's later redemption even though the later epoch was settled and its assets were reserved. Example flow:

1. User A requests a redeem in epoch 1, so `redeemRequestQueue[A] = [1]`.
2. User B also has pending redeem shares in epoch 1.
3. User A cancels, which sets `_redeemRequests[1][A].shares = 0` while leaving queue entry `1` at A's cursor.
4. Epoch 1 settles because user B's shares remain pending.
5. User A makes a new redeem request in epoch 2, so A's queue becomes `[1, 2]`.
6. Epoch 2 settles.

At this point A's later request is claimable by direct request id, but A cannot claim it through `redeem()` or `withdraw()` because the FIFO cursor is stuck on the cancelled zero-share request from epoch 1.

The affected controller's assets from the later redemption are locked in the vault and remain included in `totalClaimableRedeemAssets`, which also reduces `availableIdleAssetsForStrategy()`. There is no public function that lets the controller advance past the cancelled queue head, because `_advanceRedeemClaimCursorIfClaimed()` is only reached after a successful claim.

Recommendations:

When cancelling the current redeem request, remove the current epoch id from the controller's queue if it is the queue tail, or advance the controller cursor when the cancelled request is the current cursor. Additionally, make claim resolution robust by skipping settled zero-share queue entries before returning the current redeem request, so stale cancelled requests cannot block later claimable epochs.

Venzo: Resolved with `@88213222c0 ...`

Zenith: Verified.

4.2 Medium Risk

A total of 12 medium risk findings were identified.

[M-1] The calculation of assetsPerShare and activeSupply in _consumeSettlementReport is incorrect

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target

- PricingModule.sol

Description:

In `_consumeSettlementReport`, the previously reported `assetsPerShare` is reused, and the `active supply` is derived from it.

```
function _consumeSettlementReport(uint256 reportId, uint256 epochOpenedAt)
    internal
    view
    virtual
    returns (uint256 navAssets, uint256 assetsPerShare,
    uint256 externalValueAssets, uint256 activeSupply)
{
    assetsPerShare = report_.assetsPerShare;
    externalValueAssets = report_.externalValueAssets;
    activeSupply = _deriveActiveSupply(navAssets, assetsPerShare);
}
```

However, this can become incorrect during `epoch pricing`. For example, if there are two `epoches` and the `first epoch` is settled, additional `fee shares` are minted, which reduces the `share price`. However, when settling the second `epoch` with the same `report`, the previously snapshot `assetsPerShare` is still used. This leads to an incorrect `active supply` calculation and can make the `withdrawal` workflow incorrect.

Recommendations:

When settling an epoch, `assetsPerShare` can be calculated for that specific `epoch`.

Venzo: Resolved with `@7b6496a526 ...`

Zenith: Verified.

[M-2] The time gap between pricing and settlement can lead to unexpected changes in the share price

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target

- AsyncRedeemVault.sol

Description:

When an `epoch` is `priced`, the `claimable assets`, `shares`, and `assetsPerShares` are snapshoted.

```
function _priceRedeemEpoch(uint256 epochId)
    internal returns (uint256 assets) {
    ...
    epoch.settledAssetsPerShare = assetsPerShare;
    epoch.pricedAssets = assets;
    epoch.pricedNetShares = netShares;
}
```

However, these values are still included in the `active ones`. If `profits` or `losses` occur before the `epoch` is settled, they also affect these `snapshoted shares`. At settlement time, the previously `snapshoted assetsPerShares` is used.

```
function _settleRedeemEpoch(uint256 epochId)
    internal returns (uint256 assets, uint256 feeShares) {
    assets = epoch.pricedAssets;
    uint256 netShares = epoch.pricedNetShares;
    feeShares = shares - netShares;

    epoch.settledAt = block.timestamp;
    epoch.status = RedeemEpochStatus.Settled;
    epoch.totalPendingShares = 0;
    lastSettledRedeemEpochId = epochId;
@->     totalClaimableRedeemAssets += assets;
@->     totalClaimableRedeemNetShares += netShares;
    _setActiveNavAssets(epoch.reportId, cachedActiveNav.assets - assets);
```

```
        emit RedeemEpochSettled(epochId, epoch.reportId, shares, assets,
        epoch.settledAssetsPerShare);
    }
```

As a result, any changes that occurred after `pricing` are immediately reflected in the remaining `active shares`. This creates a window where users may benefit by `depositing` before `settlement`, as they can capture the updated valuation. Alternatively, the remaining users may incur losses from this.

Recommendations:

The `claimable assets` and `shares` should be computed at `settlement time` and immediately removed from the `active state`. `Pricing` and `settlement` can be combined into a single function.

Venzo: Resolved with `@49deaa0f5d ...`

Zenith: Verified.

[M-3] The redeem epoch should be priced using its start time rather than its closing time

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target

- AsyncRedeemVault.sol

Description:

Only closed `epochs` can be priced. In `_priceRedeemEpoch()`, if the `epoch` is still open, `_closeRedeemEpoch()` is invoked. This updates `closedAt` to the current `block timestamp`.

```
function _priceRedeemEpoch(uint256 epochId)
    internal returns (uint256 assets) {
    RedeemEpochState storage epoch = _redeemEpochs[epochId];
    if (epoch.status == RedeemEpochStatus.Open) _closeRedeemEpoch(epochId);
    if (epoch.status ≠ RedeemEpochStatus.Closed) revert InvalidEpoch();
    if (epochId ≠ lastPricedRedeemEpochId + 1) revert InvalidEpoch();

    uint256 shares = epoch.totalPendingShares;
    if (shares == 0) revert EmptyEpoch();

    uint256 reportId = reportOracle.latestReportId();
    (uint256 navAssets, uint256 assetsPerShare, uint256 externalValueAssets,
    uint256 activeSupply) =
        _consumeSettlementReport(reportId, epoch.closedAt);
```

However, `_consumeSettlementReport()` subsequently reverts because the `computed timestamp` of the latest `report` is earlier than the current `block timestamp` (`closedAt`).

```
function _consumeSettlementReport(uint256 reportId, uint256 epochOpenedAt)
    internal
    view
    virtual
    returns (uint256 navAssets, uint256 assetsPerShare,
    uint256 externalValueAssets, uint256 activeSupply)
{
    IReportOracle.Report memory report_ = reportOracle.report(reportId);
```

```
    if (!report_.exists) revert InvalidReport();
    if (epochOpenedAt ≠ 0 && report_.computedAt < epochOpenedAt)
    revert ReportBeforeEpoch();
```

As a result, `pricing` an open `epoch` always fails.

Recommendations:

```
function _priceRedeemEpoch(uint256 epochId)
    internal returns (uint256 assets) {
    RedeemEpochState storage epoch = _redeemEpochs[epochId];
    if (epoch.status == RedeemEpochStatus.Open) _closeRedeemEpoch(epochId);
    if (epoch.status ≠ RedeemEpochStatus.Closed) revert InvalidEpoch();
    if (epochId ≠ lastPricedRedeemEpochId + 1) revert InvalidEpoch();

    uint256 shares = epoch.totalPendingShares;
    if (shares == 0) revert EmptyEpoch();

    uint256 reportId = reportOracle.latestReportId();
    (uint256 navAssets, uint256 assetsPerShare, uint256 externalValueAssets,
    uint256 activeSupply) =
        _consumeSettlementReport(reportId, epoch.closedAt);
        _consumeSettlementReport(reportId, epoch.openedAt);
```

or

```
function _priceRedeemEpoch(uint256 epochId)
    internal returns (uint256 assets) {
    RedeemEpochState storage epoch = _redeemEpochs[epochId];
    if (epoch.status == RedeemEpochStatus.Open) _closeRedeemEpoch(epochId);
    if (epoch.status ≠ RedeemEpochStatus.Closed) revert InvalidEpoch();
    if (epochId ≠ lastPricedRedeemEpochId + 1) revert InvalidEpoch();

    uint256 shares = epoch.totalPendingShares;
    if (shares == 0) revert EmptyEpoch();

    uint256 reportId = reportOracle.latestReportId();
    (uint256 navAssets, uint256 assetsPerShare, uint256 externalValueAssets,
    uint256 activeSupply) =
        _consumeSettlementReport(reportId, epoch.closedAt);
```

Venzo: Resolved with @c2422573a3 ...

Zenith: Verified.

[M-4] Redeem settlement can regress the NAV cache

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target

- AsyncRedeemVault.sol
- PricingModule.sol

Description:

`AsyncRedeemVault._priceRedeemEpoch()` stores the oracle `reportId` used to price a redeem epoch and writes that report's active NAV into `cachedActiveNav`. Later, `_settleRedeemEpoch()` subtracts the priced redeem assets from the current `cachedActiveNav.assets`, but writes the result back with `epoch.reportId`.

If a newer oracle report or a deposit updates `cachedActiveNav` between pricing and settlement, settlement can relabel a newer cache value with an older report id. After that, `PricingModule._latestActiveNavAssets()` sees `reportOracle.latestReportId() > cachedActiveNav.reportId` and returns the newer report's raw `activeNavAssets`, ignoring the deposit and settlement adjustments that were already reflected in the cache before it was regressed.

This can cause later deposits and mints to price shares from an understated or overstated active NAV. In the underpricing case, a depositor can receive too many shares and dilute existing shareholders until a later report or state transition corrects the cache.

Recommendations:

Do not allow settlement of an old priced epoch to overwrite a newer cache with an older report id. Require the cache to still correspond to `epoch.reportId` before settlement, or rebase settlement against the freshest available NAV and preserve the newest report id when writing `cachedActiveNav`.

Venzo: Resolved with `@11ba191eca ...`

Zenith: Verified.

[M-5] Active supply is under-derived for non-18-decimal assets

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: High

Target

- PricingModule.sol

Description:

`Base7540Vault._deriveAssetsPerShare()` derives the vault price by first normalizing `navAssets` from asset decimals into 18-decimal share units, then dividing by `activeSupply`. For assets with fewer than 18 decimals, this includes the `shareUnit / assetUnit` decimal offset. `PricingModule._consumeSettlementReport()` reconstructs `activeSupply` from the report NAV and assets-per-share by calling `_deriveActiveSupply()`, but the base implementation only computes `navAssets * PRICE_SCALE / assetsPerShare`. It does not apply the inverse decimal normalization used by `_deriveAssetsPerShare()`, so a 6-decimal asset such as USDC derives supply in asset units instead of share units. For example, a vault with `1,000e6` NAV, `1,000e18` active shares, and a `1e18` price derives `1,000e6` active supply instead of `1,000e18`. This recalculation is also unnecessary because the finalized report already stores `report_.activeShareSupply`. Re-deriving the value from `activeNavAssets` and `assetsPerShare` reintroduces the same decimal-basis mismatch and can diverge from the supply value that was originally approved in the report. Redeem settlement passes this under-derived supply into `_mintManagementFee()` and `_mintPerformanceFee()`. Fee shares are therefore minted against a near-zero supply for non-18-decimal assets, materially undercharging management and performance fees. The performance fee path is especially affected because the high-water mark can still advance even though the corresponding fee shares were not minted.

Recommendations:

Override `_deriveActiveSupply()` in `Base7540Vault` with the inverse of `_deriveAssetsPerShare()`, applying `shareUnit / assetUnit` before dividing by `assetsPerShare`. Add coverage for at least one 6-decimal asset to assert that `_consumeSettlementReport()` returns supply in share units and that settlement fee accrual uses the expected active supply.

Venzo: Resolved with `@726778650d ...`

Zenith: Verified.

[M-6] Share transfers do not validate blocked senders

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: High

Target

- Base7540Vault.sol

Description:

`Base7540Vault._update()` enforces the vault pause state, `shareTransferEnabled`, and the configured access policy for ordinary investor-to-investor share transfers. However, the policy check only validates `to` via `_requireAllowed(to)` and never validates `from`.

This is inconsistent with the deposit and mint paths, which require `_requireAllowedPair(msg.sender, receiver)`, and with the pair-based access model exposed by `AccessController`. When the vault is in `AccessMode.Policy`, an account that previously received shares can later be removed from the allowlist or placed on a denylist, but it can still transfer those shares to any currently allowed recipient.

As a result, the access policy cannot reliably immobilize restricted share holders. A blocked holder can move shares to a compliant or colluding account, which can then continue normal vault flows such as redeeming or transferring the shares onward.

Recommendations:

Validate both sides of ordinary share transfers. Replace `_requireAllowed(to)` with `_requireAllowedPair(from, to)`.

Venzo: Resolved with `@16182c01bc ...`

Zenith: Verified.

[M-7] Deposits can use expired cached NAV

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target

- PricingModule.sol

Description:

`PricingModule._latestActiveNavAssets()` enforces `maxReportAge` only when it reads a newer oracle report. If the latest report is stale, or if `cachedActiveNav` already points to the latest report, the function falls through and returns `cachedActiveNav.assets` without validating whether the report underlying that cache has also expired.

`Base7540Vault.deposit()` and `Base7540Vault.mint()` consume this value through `totalAssets()`, `trustedAssetsPerShare()`, `previewDeposit()`, and `previewMint()`. Once a fresh report has been cached by a prior deposit or settlement, the vault can continue issuing shares from that cached NAV after the oracle freshness window has elapsed.

If off-chain NAV has moved since the cached report, new deposits can be priced incorrectly. When the true NAV has increased, a depositor can receive too many shares and dilute existing shareholders; when the true NAV has decreased, new depositors can overpay. The same stale NAV is also used by capacity checks, so deposit limits can be evaluated against an expired accounting baseline.

Recommendations:

Track or recover the timestamp for the report represented by `cachedActiveNav`, and enforce `maxReportAge` before using cached NAV in price-sensitive paths. If the latest report and the cached report are both stale, `deposit()`, `mint()`, preview functions, and settlement pricing should revert or enter an explicitly documented emergency mode instead of silently pricing from the expired cache.

Venzo: Resolved with `@a4e32845d9 ...`

Zenith: Verified.

[M-8] The openedAt should be reset when all epochs are cancelled

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target

- AsyncRedeemVault.sol

Description:

When requesting a redeem `epoch`, `openedAt` is updated.

```
function _requestRedeemIntoEpoch(address controller, uint256 shares)
    internal returns (uint256 requestId) {
    if (epoch.openedAt == 0) epoch.openedAt = block.timestamp;
    epoch.totalPendingShares += shares;
}
```

However, when redemption requests are cancelled, `openedAt` is not reset.

```
function _cancelRedeemRequest(address controller)
    internal returns (uint256 requestId, uint256 shares) {
    request.shares = 0;
    _redeemEpochs[requestId].totalPendingShares -= shares;

    emit RedeemRequestCanceled(requestId, controller, shares);
}
```

As a result, any user can manipulate `openedAt` by requesting and cancelling `redemptions` for a newly `opened epoch`. This can be used to bypass the check described below.

```
function _closeRedeemEpoch(uint256 epochId) internal {
    if (block.timestamp < epoch.openedAt + redeemEpochDuration)
    revert EpochDurationNotElapsed();
}
```

Recommendations:

```
function _cancelRedeemRequest(address controller)
    internal returns (uint256 requestId, uint256 shares) {
    request.shares = 0;
    _redeemEpochs[requestId].totalPendingShares -= shares;
    if (_redeemEpochs[requestId].totalPendingShares == 0) {
        _redeemEpochs[requestId].openedAt = 0;
    }
    emit RedeemRequestCanceled(requestId, controller, shares);
}
```

Venzo: Resolved with `@f53d2eec86 ...`

Zenith: Verified.

[M-9] There could be a conflict between the strategy manager and the oracle report

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target

- OffchainStrategyManager.sol

Description:

There is a time gap between when the `report` is computed and when it is finalized. During this period, the `strategy manager` may move funds into or out of the `vault` after the computation time, and these changes are not reflected in the `report`. As a result, once the `report` is finalized, the calculated `active NAV` may be incorrect.

Recommendations:

Record the last `asset movement timestamp` in the `strategy manager`. If the `report`'s `computed timestamp` is earlier than this value at the time of submission, the `report` should be ignored.

Venzo: Resolved with `@533f5a440c ...`

Zenith: Verified.

[M-10] The calculation of the management fee and performance fee is incorrect

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: High

Target

- AsyncRedeemVault.sol

Description:

Management and performance fees are calculated only during epoch settlement.

```
function settleRedeemEpoch(uint256 epochId)
    public
    virtual
    onlyRole(Roles.SETTLEMENT_ROLE)
    returns (uint256 assets)
{
    uint256 feeShares;
    (assets, feeShares) = _settleRedeemEpoch(epochId);
    if (totalClaimableRedeemAssets >
    IERC20(asset()).balanceOf(address(this)))
    revert InsufficientIdleAssets();
    RedeemEpochState storage epoch = _redeemEpochs[epochId];
    _mintManagementFee(epoch.activeShareSupply,
    epoch.settledAssetsPerShare);
    _mintPerformanceFee(epoch.activeShareSupply,
    epoch.settledAssetsPerShare);
    _collectProtocolFeeShares(feeShares);
}
```

- If a long time passes after an epoch is settled, and the total shares increase significantly due to deposits, then when the next epoch is settled, the fee calculation includes these newly added shares as if they had existed for the entire period since the last settlement.
- If two epochs are priced using the same latest report, the snapshot of active supply for the second epoch becomes incorrect once the first epoch is settled. This incorrect active supply is then used in the fee calculation when the second epoch is settled.

Furthermore, `settled epochs` are not affected by `management` and `performance fees`, even though their `shares` are still included in the `active share supply`.

Recommendations:

These `fees` should be recalculated whenever there are changes to the `total supply`, such as `deposits` or `settlements`. Also the current `total supply` should be used instead of the snapshot of `active supply` in the `epoch`. And the calculation of `claimable assets` and `shares` should be performed after accounting for these `fees`.

Venzo: Resolved with `@8eca1fef1f...`

Zenith: Verified.

[M-11] The rounding direction is incorrect when depositing into the vault

SEVERITY: Medium	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: High

Target

- Base7540Vault.sol

Description:

When `depositing` or `minting` in the `vault`, calculations should favor the `vault`. For this reason, `previewDeposit` rounds shares down.

```
function previewDeposit(uint256 assets)
    public view virtual override returns (uint256 shares) {
    uint256 grossShares = convertToShares(assets);
    return grossShares - _calculateFee(grossShares,
    feeManager.entryFeeRate());
}
```

However, `trustedAssetsPerShare` is always `rounded down`, and `shares` are computed by dividing `assets` by `assetsPerShare`.

```
function convertToShares(uint256 assets)
    public view virtual override returns (uint256 shares) {
    return _convertToSharesAt(assets, trustedAssetsPerShare());
}

function _convertToSharesAt(uint256 assets, uint256 assetsPerShare,
    Math.Rounding rounding)
    internal
    view
    virtual
    returns (uint256 shares)
{
    return assets.mulDiv(_shareUnit(), _assetUnit(),
    rounding).mulDiv(PRICE_SCALE, assetsPerShare, rounding);
}
```

Recommendations:

For correct design, `assetsPerShare` should be `rounded up` during `deposit` and `mint` operations to ensure proper `vault` protection.

Venzo: Resolved with `@1ca4b6908d ...`

Zenith: Verified.

[M-12] The fee is not correct minted when settling epochs

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: High

Target

- AsyncRedeemVault.sol

Description:

When `settling epochs`, `fees` are minted to the `protocol` and the corresponding `net shares` are added to `totalClaimableRedeemNetShares`, which is not included in the `active supply`.

```
function _settleRedeemEpoch(uint256 epochId)
    internal returns (uint256 assets, uint256 feeShares) {
    assets = epoch.pricedAssets;
    uint256 netShares = epoch.pricedNetShares;
    feeShares = shares - netShares;

    totalClaimableRedeemAssets += assets;
    totalClaimableRedeemNetShares += netShares;
}

function settleRedeemEpoch(uint256 epochId)
    public
    virtual
    onlyRole(Roles.SETTLEMENT_ROLE)
    returns (uint256 assets)
{
    uint256 feeShares;
    (assets, feeShares) = _settleRedeemEpoch(epochId);
    _collectProtocolFeeShares(feeShares);
}
```

However, the corresponding `fee shares` still remain in `address(this)`, which leads to them being counted twice and temporarily reducing the `share price`.

These extra `fee shares` in `address(this)` are later removed when users `withdraw` their `requests`, by burning both `net shares` and the associated `fee shares`.

```
function withdraw(uint256 assets, address receiver, address controller)
    public
    virtual
    override
    returns (uint256 shares)
{
    _requireNotPaused();
    _requireControllerOrOperator(controller, msg.sender);
    _requireAllowedPair(receiver, controller);
    (shares,) = _withdrawRedeemFromEpoch(controller, assets);

    _burn(address(this), shares);
    IERC20(asset()).safeTransfer(receiver, assets);

    emit Withdraw(msg.sender, receiver, controller, assets, shares);
}
```

As a result, the `share price` increases immediately after `withdrawal`.

Recommendations:

```
function settleRedeemEpoch(uint256 epochId)
    public
    virtual
    onlyRole(Roles.SETTLEMENT_ROLE)
    returns (uint256 assets)
{
    _collectProtocolFeeShares(feeShares);
}

function withdraw(uint256 assets, address receiver, address controller)
    public
    virtual
    override
    returns (uint256 shares)
{
    (shares,) = _withdrawRedeemFromEpoch(controller, assets);
    (shares, feeShares) = _withdrawRedeemFromEpoch(controller, assets);

    _burn(address(this), shares);
    _burn(address(this), shares - feeShares);
      _collectProtocolFeeShares(feeShares);
```

```
}

function redeem(uint256 shares, address receiver, address controller)
    public
    virtual
    override
    returns (uint256 assets)
{
    (assets,) = _claimRedeemFromEpoch(controller, shares);
    (assets, feeShares) = _claimRedeemFromEpoch(controller, shares);

    _burn(address(this), shares);
    _burn(address(this), shares - feeShares);
     _collectProtocolFeeShares(feeShares);
}
```

Venzo: Resolved with @35be756862 ...

Zenith: Verified.

4.3 Low Risk

A total of 12 low risk findings were identified.

[L-1] Redemption pricing excludes accrued management and performance fees

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Medium

Target

- AsyncRedeemVault.sol

Description:

`AsyncRedeemVault._priceRedeemEpoch()` fixes redeeming users' asset payouts using the current `assetsPerShare` before accrued management and performance fees are minted. `_settleRedeemEpoch()` then reserves those priced assets and subtracts them from active NAV, and only after that does `settleRedeemEpoch()` call `_mintManagementFee()` and `_mintPerformanceFee()`. Because the fee shares are minted after the redeem assets have already been fixed, the dilution from accrued management and performance fees is applied only to the remaining active shareholders. The fee calculation still uses the epoch's pre-settlement active supply, so remaining holders can effectively bear the fee burden attributable to shares that are exiting in the same epoch. For example, with 1,000 shares, 1,100 assets, a high-water mark of 1.0, and a 20% performance fee, a 500-share redemption is priced at 550 assets before the performance fee is minted. If fees were accrued before pricing the redemption, the post-fee price would be lower and the same redeemer would receive roughly 540 assets, leaving the missing fee value to be absorbed by the remaining holders.

Recommendations:

Accrue management and performance fees before pricing redeem epochs, or compute redeem payouts from a post-fee `assetsPerShare`. Fee accrual should update supply, the high-water mark, and the management-fee timestamp before any redemption assets are fixed.

Venzo: Resolved with `@ecaf18fd36 ...`

Zenith: Verified.

[L-2] Redeem max views ignore pause and access policy state

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Medium

Target

- AsyncRedeemVault.sol

Description:

`AsyncRedeemVault.maxWithdraw()` and `maxRedeem()` return the controller's current claimable redeem assets and shares directly from `_maxClaimableRedeem()`. They do not account for the global pause state or whether the controller still satisfies the configured access policy.

The executable claim paths, `withdraw()` and `redeem()`, both call `_requireNotPaused()` and `_requireAllowedPair(receiver, controller)`. Therefore, `maxWithdraw()` and `maxRedeem()` can advertise a non-zero amount even when the corresponding claim would revert because the vault is paused or the controller is no longer allowed.

This does not bypass the checks in the state-changing functions, but it makes the ERC-4626/7540 limit views inconsistent with actual claimability and can mislead routers, frontends, and integrations that use `maxWithdraw()` or `maxRedeem()` as preflight checks.

Recommendations:

Return zero from `maxWithdraw()` and `maxRedeem()` when the vault is paused or when the controller fails the current access policy. If receiver-specific access checks cannot be represented by these views, document that limitation and at least gate on controller eligibility and global pause state.

Venzo: Resolved with `@09c59acbec ...` We updated the async claim limit views to return 0 when the vault is globally paused or when the controller no longer passes the current access policy. This was applied to maxWithdraw() / maxRedeem().One limitation remains: these max view functions only take controller, while the actual claim functions also check the receiver/controller pair. So receiver-specific access restrictions cannot be fully represented by these views and are still enforced in the state-changing functions.

Zenith: Verified.

[L-3] `maxDeposit()` and `maxMint()` can return unusable limits

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Medium

Target

- Base7540Vault.sol

Description:

`deposit()` rejects any asset amount below `minDepositAssets`, and `mint()` applies the same minimum after converting the requested shares to assets with `previewMint()`. However, `maxDeposit()` only accounts for pause state, receiver access, and the remaining `maxTotalAssets` capacity. If the remaining capacity is non-zero but below `minDepositAssets`, `maxDeposit()` returns that unusable remainder and `maxMint()` converts it to shares through `previewDeposit()`.

This makes the ERC-4626 max functions overstate the amount that can actually be supplied. For example, with `maxTotalAssets = 100 ether`, `minDepositAssets = 10 ether`, and `totalAssets() = 95 ether`, `maxDeposit()` returns `5 ether` and `maxMint()` returns `5 ether`, but both `deposit(5 ether, receiver)` and `mint(5 ether, receiver)` revert with `BelowMinimumAmount`.

Integrators that rely on `maxDeposit()` or `maxMint()` to size valid transactions can submit transactions that are guaranteed to revert. The impact is limited to denial of service and incorrect integration state around the remaining-capacity boundary.

Recommendations:

Make `maxDeposit()` return zero when the remaining finite capacity is below the enforced minimum deposit amount, so `maxMint()` inherits the same behavior.

```
  uint256 currentAssets = totalAssets();
  if (currentAssets >= maxTotalAssets) return 0;
return maxTotalAssets - currentAssets;
uint256 remainingAssets = maxTotalAssets - currentAssets;
if (minDepositAssets ≠ 0 && remainingAssets < minDepositAssets) return 0;
return remainingAssets;
```

Venzo: Resolved with `@e87826bd8f ...`

Zenith: Verified.

[L-4] `maxDeposit()` and `maxMint()` do not mirror caller access checks

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Medium

Target

- Base7540Vault.sol

Description:

`deposit()` and `mint()` both call `_requireAllowedPair(msg.sender, receiver)`, so a synchronous deposit only succeeds when the caller and receiver satisfy the configured access policy. In contrast, `maxDeposit(receiver)` delegates to `_syncDepositLimitBlocks(receiver)`, which only checks `accessPolicy.canAccess(address(this), receiver, msg.data)`. `maxMint(receiver)` depends on `maxDeposit(receiver)`, so it inherits the same receiver-only access view.

When a static allowlist or denylist policy is used, a blocked caller can receive a non-zero `maxDeposit()` or `maxMint()` value for an allowed receiver, even though the actual `deposit()` or `mint()` call reverts in `_requireAllowedPair()`. With signature-based authorization policies, the mismatch can also appear in the opposite direction: the max functions use `canAccess()`, while the state-changing deposit path uses `validateAccess()` or `validateAccessPair()` with signed calldata.

The result is that the max functions are not a faithful view of the synchronous deposit and mint acceptance rules. This can break routers, frontends, and off-chain systems that use the max functions as preflight checks before submitting user transactions.

Recommendations:

For static policies, make the max path check the same caller-and-receiver pair as the state-changing path. If signed authorization policies remain supported, document that ERC-4626 max functions cannot validate a signed deposit payload, or add a dedicated preflight helper that accepts the intended call context.

```
function maxDeposit(address receiver)
    public view virtual override returns (uint256 maxAssets) {
    if (_syncDepositLimitBlocks(receiver)) return 0;
```

```
    if (_syncDepositLimitBlocks(msg.sender, receiver)) return 0;
     if (maxTotalAssets == 0) return type(uint256).max;

     uint256 currentAssets = totalAssets();
     if (currentAssets >= maxTotalAssets) return 0;
     return maxTotalAssets - currentAssets;
 }

function _syncDepositLimitBlocks(address receiver) internal view returns (
    bool) {
function _syncDepositLimitBlocks(
     address caller, address receiver) internal view returns (bool) {
     if (paused() || depositsPaused) return true;
     if (accessMode == AccessMode.Open) return false;
     if (address(accessPolicy) == address(0)) return true;
    return !accessPolicy.canAccess(address(this), receiver, msg.data);
    if (caller == receiver) return !accessPolicy.canAccess(address(this),
        receiver, msg.data);

    return !accessPolicy.canAccessPair(address(this), caller, receiver, msg.
        data);
 }
```

Venzo: Resolved with @ef6160cd3f...

Zenith: Verified.

[L-5] The initialNavAssets should be stored in a cache

SEVERITY: Low	IMPACT: Medium
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- ReportOracle.sol

Description:

In the `ReportOracle` constructor, `initialNavAssets` is reported but not cached.

```
constructor(address admin, uint64 initialMaxReportAge,
    uint256 initialNavAssets, uint64 initialComputedAt) {
    if (admin == address(0)) revert ZeroAddress();
    if (initialMaxReportAge == 0) revert InvalidReportValue();

    _grantRole(DEFAULT_ADMIN_ROLE, admin);
    _grantRole(Roles.ORACLE_ADMIN_ROLE, admin);

    _setOracleReporter(admin, true);

    maxReportAge = initialMaxReportAge;
    _finalizeInitialReport(initialNavAssets, initialComputedAt, admin);
}
```

As a result, if the `first deposit` or subsequent `reporting` occurs after this initial value becomes outdated, `initialNavAssets` is not taken into account.

Recommendations:

This `initialNavAssets` should be cached as well.

Venzo: Acknowledged. The `initialNavAssets` is finalized as the first oracle report, but the vault-side active NAV cache is not initialized from that report. If no settlement updates the cache before the initial report becomes stale, the vault may fall back to an empty cached NAV.

[L-6] Unclaimable dust shares may permanently remain in the vault

SEVERITY: Low	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- AsyncRedeemVault.sol

Description:

An `epoch` can include `withdrawal requests` from multiple users, and `fee calculations` are always `rounded up`. During `epoch settlement`, `fees` are applied to the sum of all individual `shares`. However, when users `withdraw`, `fees` are applied per individual `share`. As a result, the sum of per-user `fees` may exceed the `fee` calculated on the aggregated `shares`. This discrepancy can prevent `totalClaimableRedeemNetShares` from fully reaching zero, leaving residual dust amounts that may remain permanently in the `vault`.

Or revert can happen below.

```
function _withdrawRedeemFromEpoch(address controller, uint256 assets)
    internal
    returns (uint256 shares, uint256 feeShares)
{
    request.shares = request.shares > shares ? request.shares - shares : 0;
    totalClaimableRedeemAssets -= assets;
@->    totalClaimableRedeemNetShares -= shares - feeShares;
    _advanceRedeemClaimCursorIfClaimed(controller, request);
}
```

Recommendations:

This could be acknowledged.

```
function _withdrawRedeemFromEpoch(address controller, uint256 assets)
    internal
    returns (uint256 shares, uint256 feeShares)
{
    request.shares = request.shares > shares ? request.shares - shares : 0;
```

```
    totalClaimableRedeemAssets -= assets;
    totalClaimableRedeemNetShares -= shares - feeShares;
    if (totalClaimableRedeemAssets ≥ shares - feeShares) {
        totalClaimableRedeemNetShares -= shares - feeShares;
    }
    _advanceRedeemClaimCursorIfClaimed(controller, request);
}
```

Venzo: Acknowledged. This may leave positive accounting dust in `totalClaimableRedeemNetShares` because `settlement` applies `exit fees` on aggregate `epoch shares` while `claims` apply `rounded-up fees` per `request`. The dust does not cause underflow or prevent `claims`, but it can slightly understate `activeShareSupply`. We consider the impact negligible/bounded by rounding granularity and will document it, or address it in a future accounting cleanup.

[L-7] Users can adjust their deposits based on the upcoming report

SEVERITY: Low	IMPACT: Medium
STATUS: Acknowledged	LIKELIHOOD: Medium

Target

- AsyncRedeemVault.sol

Description:

Allowed `reporters` can submit `reports`, and a `report` is finalized once the required threshold is reached.

```
function _submitReport(uint256 externalValueAssets, uint64 computedAt,
    bytes32 metadataHash, address reporter_)
    internal
    returns (uint256 reportId)
{
    hasApprovedReport[digest][reporter_] = true;
    uint256 approvalCount = ++reportApprovalCount[digest];
    uint256 quorum = oracleQuorum;
    emit ReportApproved(digest, approvalCount, quorum, externalValueAssets,
    metadataHash, computedAt, reporter_);

    if (approvalCount < quorum) return 0;

    reportId = _finalizeReport(digest, externalValueAssets, computedAt,
    metadataHash, reporter_);
}
```

After the first reporter submits a `report`, users can infer the `upcoming share price` before the `report` is finalized. If the `upcoming share price` is much higher than the `current share price`, users can make a large `deposit` before finalization. As a result, they receive `shares` at the `current exchange rate` and immediately benefit from the `higher share price` once the `report` is finalized.

Recommendations:

- Assuming the `reporters` are trusted, the following mitigation can be implemented:

When the first `report` is submitted, `deposits` into the `vault` should be paused. `Deposits` can be resumed once the `report` is finalized. Additionally, the `computedAt` value of newly submitted `reports` should be validated to ensure it never decreases relative to previously submitted `reports`.

- Alternatively, this behavior can be acknowledged as acceptable, since the resulting increase in vault `assets` benefits the `protocol`.

Venzo: Acknowledged. We added submitReportWithSignatures, allowing reporters to sign the report offchain and submit enough signatures in one transaction. This lets long-cycle reports reach quorum atomically, avoiding a public pending-report window before finalization. We also added reportSignatureDigest so reporter tooling can sign the exact digest verified by the oracle. The existing submitReport flow remains available for short-cycle or single-reporter reports.

[L-8] `_convertToAssetsAt()` truncates share value before applying price

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Medium

Target

- Base7540Vault.sol

Description:

`_convertToAssetsAt()` converts shares back to assets by first scaling the 18-decimal share amount down to the asset decimals, then applying `assetsPerShare`. This is inconsistent with the price convention used by `trustedAssetsPerShare()`, where `assetsPerShare` is a 1e18-scaled price over normalized 18-decimal asset value.

For assets with fewer than 18 decimals, the first `mulDiv()` can discard share value before the price multiplier is applied. With a 6-decimal asset, `assetsPerShare = 1.5e18`, and `shares = 1e12 - 1`, the mathematically correct floor conversion returns one raw asset unit, but the current implementation first rounds the share amount down to zero asset units and returns zero. The same premature scaling can also make ceil conversions charge more assets than necessary.

The impact is limited to small precision losses around raw asset-unit boundaries, but it affects all asset conversions that use `_convertToAssetsAt()` with sub-18 decimal assets and non-1e18 prices, including redeem and mint accounting paths.

Recommendations:

Apply `assetsPerShare` before reducing from 18-decimal share units to asset decimals. For example, compute the price-adjusted 18-decimal value first, then scale to asset units using the same rounding direction.

Venzo: Resolved with `@5512d3d63c ...`

Zenith: Verified.

[L-9] Redeem cancellation bypasses pause and current access checks

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Medium

Target

- AsyncRedeemVault.sol

Description:

`cancelRedeemRequest()` lets a controller or approved operator cancel a pending redeem request when `redeemCancellationEnabled` is true. Unlike other user-facing redeem flows, it does not check `_requireNotPaused()`, `_requireRedeemsNotPaused()`, or whether the `controller` still satisfies the current access policy.

After the request is cancelled, the vault returns the pending shares with `_transfer(address(this), controller, cancelledShares)`. This transfer also skips the ordinary share-transfer checks in `Base7540Vault._update()` because the sender is the vault itself. Consequently, cancellation can return shares to a controller during an emergency pause, while redeems are paused, or after that controller has been blocked by the configured access policy.

This weakens the protocol's ability to freeze user-facing share movement during emergency or compliance actions. The issue is limited to pending redeem requests and requires cancellation to be enabled, but once enabled the cancellation path remains available independently of the current pause and access state.

Recommendations:

Apply the same operational gates expected for user-facing redeem flows before cancellation. At minimum, call `_requireNotPaused()` and `_requireAllowed(controller)` before returning shares. If cancellations should remain available while `redeemsPaused` is true, document that exception explicitly. Otherwise, also require `_requireRedeemsNotPaused()`.

Venzo: Resolved with `@4abb233dd6 ...`

Zenith: Verified.

[L-10] The highWaterMarkAssetsPerShare monotonically increases over time

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- FeeManager.sol

Description:

`highWaterMarkAssetsPerShare` always increases in `accruePerformanceFee` and is not reset, even when all `shares` have been withdrawn.

```
function accruePerformanceFee(uint256 activeShareSupply,
    uint256 assetsPerShare)
    external
    onlyVault
    returns (uint256 feeShares)
{
    uint256 oldHighWaterMark = highWaterMarkAssetsPerShare;
    feeShares =
        FeeMath.performanceFeeShares(activeShareSupply, assetsPerShare,
    oldHighWaterMark, performanceFeeRate);

    if (assetsPerShare > oldHighWaterMark) {
        highWaterMarkAssetsPerShare = assetsPerShare;
        emit HighWaterMarkUpdated(oldHighWaterMark, assetsPerShare);
    }

    emit FeeAccrued(FEE_TYPE_PERFORMANCE, feeShares, activeShareSupply,
    assetsPerShare, oldHighWaterMark, 0);
}
```

Recommendations:

This value should be reset when the `total supply` drops to a dust amount or reaches zero.

Venzo: Resolved with `@c035658cc5 ...`

Zenith: Verified.

[L-11] The share token should not be rescuable

SEVERITY: Low	IMPACT: High
STATUS: Resolved	LIKELIHOOD: Low

Target

- Base7540Vault.sol

Description:

The `share token` can be rescued now.

```
function rescueToken(address token, address to, uint256 amount)
    external virtual onlyRole(DEFAULT_ADMIN_ROLE) {
    if (token == asset()) revert CannotRescueVaultAsset();
    if (to == address(0)) revert ZeroAddress();
    IERC20(token).safeTransfer(to, amount);
}
```

Recommendations:

```
function rescueToken(address token, address to, uint256 amount)
    external virtual onlyRole(DEFAULT_ADMIN_ROLE) {
    if (token == asset()) revert CannotRescueVaultAsset();
    if (token == address(this)) revert CannotRescueVaultAsset();
    if (to == address(0)) revert ZeroAddress();
    IERC20(token).safeTransfer(to, amount);
}
```

Venzo: Resolved with `@5f8ee70899 ...`

Zenith: Verified.

[L-12] An older report can be recorded as the latest report

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- ReportOracle.sol

Description:

When `submitting` a report, `computedAt` is not compared against the `computedAt` of the most recently stored report.

```
function _submitReport(uint256 externalValueAssets, uint64 computedAt,
    bytes32 metadataHash, address reporter_)
    internal
    returns (uint256 reportId)
{
    if (computedAt > block.timestamp) revert InvalidReportValue();
    _requireReportFresh(computedAt);

    bytes32 digest = _reportDigest(externalValueAssets, computedAt,
    metadataHash);
    reportId = finalizedReportId[digest];
}
```

As a result, an older report can be recorded as the latest if it is submitted later.

Recommendations:

When `submitting` a report, its `computedAt` value should be greater than the `computedAt` of the currently latest report.

Venzo: Resolved with `@6ba590003e ...`

Zenith: Verified.

4.4 Informational

A total of 9 informational findings were identified.

[I-1] Cancelled redeems can return shares without current transfer eligibility

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- AsyncRedeemVault.sol

Description:

`cancelRedeemRequest()` returns cancelled shares with `_transfer(address(this), controller, cancelledShares)`. Because the transfer is sent from the vault itself, `Base7540Vault._update()` skips the ordinary share-transfer branch and does not apply `_requireShareTransferEnabled()` or `_requireAllowed(controller)`.

The request authorization check only proves that the caller is the controller or an approved operator. It does not prove that the controller is still allowed by the current access policy at cancellation time. If a controller is removed from the policy after opening a redeem request, cancellation can still move the shares back to that controller.

This does not let an arbitrary account steal shares, but it weakens the access policy's ability to control where vault shares can be held after policy changes.

Recommendations:

Decide whether redeem cancellation should be allowed to return shares to controllers that no longer satisfy the access policy. If not, call `_requireAllowed(controller)` before transferring shares back. If this exception is intentional, document it alongside the cancellation flow.

Venzo: Resolved with `@d5f38fb253 ...`

Zenith: Verified.

[I-2] If settlement is not executed for a long period, it may cause future settlements to be reverted

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- FeeMath.sol

Description:

If settlement is not executed for a long period, a revert could happen below.

```
function accrueManagementFee(uint256 activeShareSupply,
    uint256 assetsPerShare)
    external
    onlyVault
    returns (uint256 feeShares)
{
    uint256 elapsed = block.timestamp - lastManagementFeeAccruedAt;
    lastManagementFeeAccruedAt = block.timestamp;
    feeShares = FeeMath.managementFeeShares(activeShareSupply,
    managementFeeRate, elapsed);
    emit FeeAccrued(FEE_TYPE_MANAGEMENT, feeShares, activeShareSupply,
    assetsPerShare, 0, elapsed);
}

function managementFeeShares(uint256 activeShareSupply, uint256 annualRate,
    uint256 elapsed)
    internal
    pure
    returns (uint256 feeShares)
{
    if (activeShareSupply == 0 || annualRate == 0 || elapsed == 0) {
        return 0;
    }

    uint256 numerator = annualRate * elapsed;
    uint256 denominator = FEE_SCALE * YEAR;
@->    if (numerator >= denominator) revert InvalidFees();
```

```
        feeShares = activeShareSupply.mulDiv(numerator, denominator - numerator,
        Math.Rounding.Ceil);
    }
```

Recommendations:

It should be ensured that settlement occurs at least once per year.

Venzo: Acknowledged. This condition is only reachable when managementFeeRate * elapsed >= FEE_SCALE * YEAR. With the intended management fee rate, such as 2% annually, this would require roughly 50 years without accrual.

[I-3] Fee manager replacement is not emitted

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- Base7540Vault.sol

Description:

`Base7540Vault.setFeeManager()` updates the external `feeManager` address after validating that the new manager is bound to the vault, but it does not emit an event.

The fee manager controls entry, exit, management, and performance fee configuration and recipients. Without an event, offchain indexers, monitoring systems, and operational run-books cannot reliably observe fee-manager replacements from logs and must reconstruct the state by polling.

Recommendations:

Add a `FeeManagerSet(address indexed oldManager, address indexed newManager)` event and emit it after updating `feeManager`.

Venzo: Resolved with `@1469e5fe47 ...`

Zenith: Verified.

[I-4] Idle assets and cached NAV must be reconciled across reports

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- ReportOracle.sol

Description:

`ReportOracle._deriveReportValues()` derives active NAV by adding the reporter-submitted external value to `availableIdleAssetsForStrategy()`. In parallel, synchronous deposits do not wait for the next oracle report to affect pricing: `Base7540Vault.deposit()` and `mint()` call `PricingModule._increaseActiveNavAssets()`, which first reads the latest report into the NAV cache and then patches the cached active NAV upward by the deposited assets.

This creates two valid accounting paths for assets between oracle reports. The oracle report path treats externally reported strategy value and the vault's current idle assets as the authoritative NAV, while the sync-deposit path locally patches the cached NAV so deposits are priced immediately. Operations that move assets between those buckets must preserve the same total economic value. `transferAssetsToStrategy()` reduces idle assets without changing the cached NAV, and `settleRedeemEpoch()` reserves idle assets for withdrawals while reducing cached active NAV.

```
function transferAssetsToStrategy(address strategy, uint256 assets)
    external virtual {
    ...
    IERC20(asset()).safeTransfer(strategy, assets);
```

```
function settleRedeemEpoch(uint256 epochId)
    ...
    if (totalClaimableRedeemAssets >
    IERC20(asset()).balanceOf(address(this)))
    revert InsufficientIdleAssets();
```

Because idle assets are included in the next oracle report but deposits and settlements can also be reflected through the patched cache, management operations must ensure that the

next reported `externalValueAssets` is consistent with the vault's current idle balance and any already-patched cached NAV. Otherwise, the next report can temporarily shift price per share by double counting assets, omitting assets, or reporting strategy value that no longer matches assets moved out of idle inventory for allocation or redemption settlement.

Recommendations:

Document and enforce the reporting invariant for management operations: when a new report is submitted, `externalValueAssets + availableIdleAssetsForStrategy()` should match the intended active NAV after sync-deposit cache patches, strategy transfers, and redeem settlements. Consider adding operational checks or events around `transferAssetsToStrategy()`, `settleRedeemEpoch()`, and report submission so offchain reporting can reconcile idle assets, strategy value, and cached NAV before finalizing a report.

Venzo: Acknowledged. We will document this reporting invariant and enforce it through the curator/reporting operational runbook. No contract change is planned for this informational issue.

externalValueAssets only represents curator strategy value and explicitly excludes vault idle assets.The oracle report derives active NAV as externalValueAssets + availableIdleAssetsForStrategy(). Transfers and report submissions are operationally separated, and reports are generated only after strategy movements are finalized. The external strategy value is independently monitored/validated through Accountable DVN or Primus-like infrastructure.

[I-5] Fee rate updates can apply retroactively to unsettled fees

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- FeeManager.sol

Description:

`FeeManager.setFeeRates()` updates management and performance fee rates without first settling pending fees under the previous configuration. Management fees are accrued later using `block.timestamp - lastManagementFeeAccruedAt`, while performance fees are calculated later against the stored high-water mark.

As a result, a newly configured management fee rate can be applied to the entire elapsed period since the last accrual, not only to time after the rate change. Similarly, a changed performance fee rate can be applied to gains that occurred before the change if those gains were not settled before updating the rate.

This creates ambiguous fee accounting around governance or admin fee changes. Depending on the direction of the rate update, managers or investors can be overcharged or undercharged relative to the intended effective date.

Recommendations:

Require an explicit fee settlement step before changing management or performance fee rates. Update `lastManagementFeeAccruedAt` and the high-water mark using a fresh vault valuation under the old rates, then apply the new rates only to future elapsed time or future gains.

Venzo: Acknowledged. setFeeRates() intentionally does not crystallize pending management or performance fees before updating rates. The fee manager is expected to control the timing of fee updates relative to settlement/accrual, so a newly configured rate may apply to the next accrual, including elapsed management-fee time or uncrystallized gains since the previous high-water mark update.We agree this behavior should be made explicit. We will document the effective-date semantics for fee-rate changes and clarify that managers who want non-retroactive rate changes should settle/accrue fees before calling setFeeRates().

[I-6] Vault interface detection omits AccessControl support

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- SyncDepositAsyncRedeemVault.sol

Description:

`SyncDepositAsyncRedeemVault.supportsInterface()` overrides the inherited implementation and manually enumerates supported interfaces. It includes ERC-7540 redeem, ERC-7575, ERC-7540 operator, and ERC-165, but it does not include the `AccessControl` interface supported by the inherited `AccessController`.

The base implementation delegates to `super.supportsInterface(interfaceId)`, which would report inherited `AccessControl` support. The concrete vault's override does not delegate to `super`, so external integrations that use ERC-165 to discover role-management support can receive a false negative even though the role functions are present.

This does not change authorization enforcement, but it can break monitoring, admin tooling, registries, or integrations that rely on ERC-165 interface detection.

Recommendations:

Include the `IAccessControl` interface id in the concrete override, or make the function delegate to the inherited `supportsInterface()` chain. Apply the same pattern to other concrete vault overrides that manually enumerate interfaces.

Venzo: Resolved with `@aeebb1a624 ...` and `@dd5c635c05 ...`

Zenith: Verified.

[I-7] Settlement report timestamp parameter is misleading

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- PricingModule.sol

Description:

`AsyncRedeemVault._priceRedeemEpoch()` passes `epoch.closedAt` into `_consumeSettlementReport()`, but the receiving parameter in `PricingModule` is named `epochOpenedAt`. The shared deposit settlement path also passes `epoch.closedAt` into the same function.

The implementation currently compares `report_.computedAt` against the provided timestamp, so the behavior is aligned with requiring reports to be computed after the epoch was closed. However, the parameter name suggests a weaker check against the epoch opening time and makes the settlement invariant easy to misunderstand during maintenance.

This is an implementation clarity issue, but future changes to report freshness or epoch settlement logic could accidentally use the wrong boundary because of the misleading name.

Recommendations:

Rename the `_consumeSettlementReport()` parameter from `epochOpenedAt` to `epochClosedAt` or a neutral name such as `minComputedAt`. Update the nearby comments to state that settlement reports must be computed at or after the epoch close timestamp.

Venzo: Resolved with `@f666421298 ...`

Zenith: Verified.

[I-8] `OffchainStrategyManager` allocation can diverge from real strategy value

`SEVERITY:` Informational	`IMPACT:` Informational
`STATUS:` Acknowledged	`LIKELIHOOD:` Low

Target

- OffchainStrategyManager.sol

Description:

`OffchainStrategyManager` tracks strategy exposure through `allocation` and `totalAllocation`, but these values are only updated when assets are allocated, returned, or manually recorded. They do not represent the strategy's current mark-to-market value. If a strategy loses value, `recordReturn()` can reduce the allocation without moving assets, effectively acting as a privileged write-off rather than a return.

The opposite case is not symmetrical. If a strategy earns yield, there is no direct way to materialize that gain in the allocation state. Returning yield through `returnAssets()` reduces allocation via `_reduceAllocation()`, which makes a yield harvest look like principal repayment. Leaving the gain in the strategy keeps NAV correct only if the oracle reports it through `externalValueAssets`, but the manager's allocation caps and outstanding-allocation checks continue to use the stale allocation value.

This can cause operational limits to diverge from real strategy exposure. A profitable strategy can hold more value than its recorded allocation, while a lossy strategy can remain blocked by outstanding allocation until the privileged write-off path is used.

Recommendations:

Document that `allocation` is principal/debt accounting rather than mark-to-market exposure, or split the accounting paths explicitly. For example, add separate functions for principal returns, yield harvests, and loss write-offs, or allow `returnAssets()` to accept an explicit `allocationReduction` so token movement and allocation accounting are not forced to move one-to-one.

Venzo: Acknowledged. We agree that `allocation` and `totalAllocation` should not be interpreted as mark-to-market strategy exposure or current NAV.The intended model is cashflow accounting only: `allocation` represents the amount historically allocated out to a strategy minus the amount returned or administratively recorded as returned. The manager in-

tentionally does not classify returned assets as principal, yield, or loss recovery, and it does not attempt to track strategy performance.

Under this model, `returnAssets()` reducing allocation by the amount returned is expected behavior, regardless of whether the returned assets economically represent principal, yield, or recovered value. Similarly, `recordReturn()` is an administrative accounting path for reducing outstanding recorded allocation when the corresponding off-chain position has been settled, impaired, or otherwise accounted for outside direct token movement.

We will clarify the documentation/comments to state that `allocation` and `totalAllocation` represent outstanding recorded allocation, not mark-to-market exposure or NAV. Strategy valuation, yield, and loss recognition are handled outside this manager, where applicable.

For this reason, we consider this an informational documentation clarification rather than a protocol accounting bug.

[I-9] Redeem allowance spending remains available while share transfers are disabled

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- AsyncRedeemVault.sol

Description:

`requestRedeem()` allows a caller that is neither `owner` nor an operator for `owner` to redeem shares by spending the owner's ERC-20 share allowance. This can be valid ERC-7540 behavior, and the request still checks `_requireAllowedPair(owner, controller)` before moving shares into the vault.

However, this allowance path remains available even when `shareTransferEnabled` is false. The internal `_transfer(owner, address(this), shares)` skips the ordinary share-transfer branch in `Base7540Vault._update()` because the recipient is the vault itself, so `_requireShareTransferEnabled()` is not applied. As a result, disabling investor-to-investor share transfers does not prevent approved spenders from moving an owner's shares into a redeem request.

If `shareTransferEnabled` is intended only to block secondary transfers, this behavior is acceptable. If it is also expected to prevent third parties from exercising share allowances to move investor positions, the current implementation should make that distinction explicit.

Recommendations:

Clarify the intended semantics of `shareTransferEnabled` for allowance-based redeem requests. If disabled share transfers should also block third-party allowance spending, require `owner == msg.sender` or `isOperator[owner][msg.sender]` when `shareTransferEnabled` is false, or add a separate configuration flag for allowance-driven redeem requests.

Venzo: Resolved with `@6aed25e090 ...`

Zenith: Verified.